

ALKE WALLET v2

Diseño e implementación de una base de datos relacional

Módulo	Fundamentos de Bases de Datos Relacionales
Proyecto	Alke Wallet
Estudiante	Angélica Peña
Fecha	Septiembre de 2026

Evaluación del módulo

Documento técnico con sentencias SQL, resultados y evidencias de implementación.

Índice de contenidos

- 1. Introducción
- 2. Objetivos
- 3. Fundamentos de bases de datos relacionales
- 4. Diseño de Alke Wallet
- 5. Implementación SQL
- 6. Transaccionalidad e integridad
- 7. Consultas avanzadas y optimización
- 8. Prueba final del proyecto
- 9. Verificación de requisitos
- 10. Conclusiones
- Anexo A. Script SQL completo

1. Introducción

Alke Wallet es un proyecto orientado al diseño de una base de datos relacional para un monedero virtual. La solución debe permitir representar usuarios, monedas y transacciones, mantener la coherencia de la información y ejecutar operaciones de creación, consulta, modificación y eliminación de datos mediante SQL.

La implementación se desarrolló con MySQL 8 sobre Ubuntu. El script fue organizado en Visual Studio Code y probado de forma local, incorporando claves primarias, claves foráneas, restricciones, consultas con JOIN, operaciones DML, transacciones controladas con COMMIT y ROLLBACK, funciones de agregación, subconsultas, índices, una vista y un modelo entidad-relación.

La estructura y el nivel de detalle de este informe siguen la consigna del proyecto del módulo “Fundamentos de Bases de Datos Relacionales”, que solicita documentar las sentencias SQL empleadas y adjuntar capturas de los resultados.

2. Objetivos

2.1 Objetivo general

Diseñar e implementar una base de datos relacional para Alke Wallet que permita almacenar y gestionar usuarios, monedas y transacciones, garantizando integridad referencial, coherencia de datos y soporte para operaciones SQL.

2.2 Objetivos específicos

- Crear la base de datos AlkeWallet y definir las tablas moneda, usuario y transaccion.
- Aplicar PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL y tipos de datos adecuados.
- Insertar datos de prueba y ejecutar consultas SELECT, WHERE, JOIN y subconsultas.
- Aplicar operaciones DML mediante INSERT, UPDATE y DELETE.
- Controlar transferencias mediante START TRANSACTION, COMMIT y ROLLBACK.
- Representar el sistema mediante un diagrama entidad-relación y documentar su normalización hasta 3FN.
- Incorporar funciones de agregación, índices compuestos, ALTER TABLE y una vista de apoyo.

3. Fundamentos de bases de datos relacionales

3.1 ¿Qué es una base de datos relacional?

Una base de datos relacional organiza la información en tablas formadas por filas y columnas. Cada tabla representa una entidad y puede relacionarse con otras mediante claves primarias y claves foráneas. Este enfoque facilita la organización, la consulta y la integridad de los datos.

3.2 Tabla, fila y columna

Concepto	Aplicación en Alke Wallet
Tabla	Conjunto de registros de una entidad, por ejemplo usuario.
Columna	Característica almacenada, por ejemplo nombre, correo o saldo.
Fila	Registro individual, por ejemplo un usuario específico.

3.3 Ventajas del modelo relacional

- Organización estructurada de la información.
- Reducción de redundancia mediante normalización.
- Relación entre entidades por medio de claves.
- Control de integridad mediante restricciones.
- Consulta eficiente con SQL.
- Soporte de transacciones y propiedades ACID.

3.4 Comparación de RDBMS

La siguiente tabla es una síntesis técnica complementaria elaborada para cumplir la actividad comparativa indicada en la consigna.

RDBMS	Tipo	Características	Uso frecuente
MySQL Community	Gratuito / código abierto	SQL relacional, ampliamente utilizado y con buen soporte para aplicaciones web.	Web, educación, servidores
PostgreSQL	Gratuito / código abierto	Extensible, completo y con fuerte soporte de estándares SQL.	Empresas, análisis, web
MariaDB	Gratuito / código abierto	Derivado de MySQL y ampliamente compatible.	Web y servidores
SQLite	Gratuito / código abierto	Ligero y basado en un archivo local.	Móvil y proyectos pequeños
Oracle Database	Comercial / propietario	Orientado a sistemas empresariales de alta disponibilidad.	Grandes organizaciones
Microsoft SQL Server	Comercial / propietario	Integración con el ecosistema Microsoft y herramientas de BI.	Empresas y BI
IBM Db2	Comercial / propietario	Orientado a grandes volúmenes y entornos corporativos.	Banca e industria

3.5 Propiedades ACID

Propiedad	Descripción	Aplicación en Alke Wallet
Atomicidad	La operación se completa totalmente o se revierte.	ROLLBACK deshace el cambio de saldo cuando falla una transferencia.
Consistencia	Los datos deben permanecer válidos.	Las claves foráneas impiden referenciar usuarios inexistentes.
Aislamiento	Las transacciones se procesan sin interferencias incorrectas.	Los cambios se agrupan dentro de una transacción controlada.
Durabilidad	Los cambios confirmados permanecen almacenados.	COMMIT confirma definitivamente la transferencia.

4. Diseño de Alke Wallet

4.1 Entidades y atributos

Entidad	Clave primaria	Claves foráneas	Atributos principales
moneda	currency_id	-	currency_name, currency_symbol
usuario	user_id	currency_id	nombre, correo, contraseña, saldo, fecha_creacion
transaccion	transaction_id	sender_user_id, receiver_user_id, currency_id	importe, transaction_date

4.2 Correspondencia ER → relacional

El modelo conceptual se transformó en tres tablas relacionadas. La tabla moneda actúa como catálogo de monedas; usuario referencia una moneda mediante currency_id; y transaccion referencia al emisor, receptor y moneda.

4.3 Diagrama entidad-relación



Figura 1. Diagrama entidad-relación de Alke Wallet. Fuente: elaboración propia.

4.4 Cardinalidades

- moneda 1:N usuario: una moneda puede ser utilizada por varios usuarios.
- usuario 1:N transaccion (emisor): un usuario puede enviar múltiples transacciones.
- usuario 1:N transaccion (receptor): un usuario puede recibir múltiples transacciones.
- moneda 1:N transaccion: una moneda puede aparecer en múltiples movimientos.

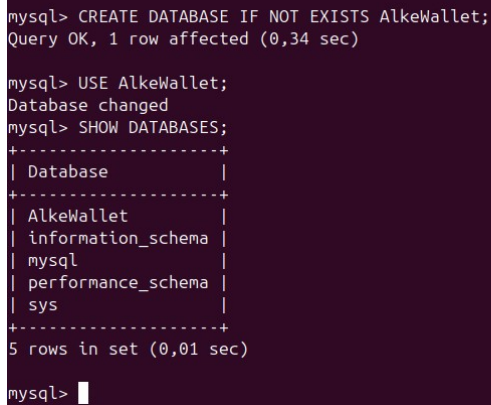
4.5 Normalización hasta Tercera Forma Normal (3FN)

El diseño mantiene atributos atómicos (1FN), cada atributo no clave depende de la clave primaria de su tabla (2FN) y la información perteneciente a otras entidades se almacena por referencia mediante claves foráneas (3FN). Por ejemplo, el nombre y símbolo de una moneda no se repiten en usuario o transaccion; se obtiene mediante currency_id. Del mismo modo, los nombres de emisor y receptor no se duplican en transaccion, sino que se recuperan mediante JOIN.

5. Implementación SQL

5.1 Creación de la base de datos

```
CREATE DATABASE IF NOT EXISTS AlkeWallet;  
USE AlkeWallet;  
SHOW DATABASES;
```



```
mysql> CREATE DATABASE IF NOT EXISTS AlkeWallet;  
Query OK, 1 row affected (0,34 sec)  
  
mysql> USE AlkeWallet;  
Database changed  
mysql> SHOW DATABASES;  
+-----+  
| Database |  
+-----+  
| AlkeWallet |  
| information_schema |  
| mysql |  
| performance_schema |  
| sys |  
+-----+  
5 rows in set (0,01 sec)  
  
mysql>
```

Figura 2. Creación y verificación de la base de datos AlkeWallet. Fuente: elaboración propia.

5.2 Creación de tablas

```
CREATE TABLE moneda (  
    currency_id INT AUTO_INCREMENT PRIMARY KEY,  
    currency_name VARCHAR(50) NOT NULL,  
    currency_symbol VARCHAR(10) NOT NULL  
);  
  
CREATE TABLE usuario (  
    user_id INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    correo VARCHAR(150) NOT NULL UNIQUE,  
    contrasena VARCHAR(255) NOT NULL,  
    saldo DECIMAL(12,2) NOT NULL DEFAULT 0.00,  
    currency_id INT NOT NULL,  
    FOREIGN KEY (currency_id) REFERENCES moneda(currency_id)  
);
```

```
CREATE TABLE transaccion (  
    transaction_id INT AUTO_INCREMENT PRIMARY KEY,  
    sender_user_id INT NOT NULL,  
    receiver_user_id INT NOT NULL,  
    currency_id INT NOT NULL,  
    importe DECIMAL(12,2) NOT NULL,  
    transaction_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (sender_user_id) REFERENCES usuario(user_id),  
    FOREIGN KEY (receiver_user_id) REFERENCES usuario(user_id),  
    FOREIGN KEY (currency_id) REFERENCES moneda(currency_id)  
);
```



```
mysql> SHOW TABLES;
+-----+
| Tables_in_AlkeWallet |
+-----+
| moneda                |
| transaccion           |
| usuario               |
+-----+
3 rows in set (0,07 sec)

mysql> DESCRIBE transaccion;
+-----+-----+-----+-----+-----+-----+
| Field                | Type          | Null | Key | Default | Extra           |
+-----+-----+-----+-----+-----+-----+
| transaction_id       | int           | NO   | PRI | NULL    | auto_increment |
| sender_user_id       | int           | NO   | MUL | NULL    |                 |
| receiver_user_id     | int           | NO   | MUL | NULL    |                 |
| currency_id          | int           | NO   | MUL | NULL    |                 |
| importe              | decimal(12,2) | NO   |     | NULL    |                 |
| transaction_date     | datetime      | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0,17 sec)
```

Figura 3. Estructura de la tabla transaccion. Fuente: elaboración propia.

5.3 Inserción y consulta de monedas

```
INSERT INTO moneda (currency_name, currency_symbol)
VALUES
('Peso Chileno', '$'),
('Dolar Estadounidense', 'US$'),
('Euro', 'EUR');

SELECT * FROM moneda;
```

```
3 rows in set (0,40 sec)

mysql> INSERT INTO moneda (currency_name, currency_symbol) VALUES ('Peso Chileno', '$'), ('Dolar Estadounidense', 'US$'), ('Euro', 'EUR');
Query OK, 3 rows affected (0,54 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM moneda;
+-----+-----+-----+
| currency_id | currency_name      | currency_symbol |
+-----+-----+-----+
| 1           | Peso Chileno       | $               |
| 2           | Dolar Estadounidense | US$            |
| 3           | Euro               | EUR             |
+-----+-----+-----+
3 rows in set (0,00 sec)

mysql>
```

Figura 4. Inserción y consulta de monedas. Fuente: elaboración propia.

5.4 Inserción de usuarios y relación con moneda

Los usuarios fueron vinculados a la moneda seleccionada mediante `currency_id`. La relación se consultó con `INNER JOIN` para obtener el nombre de la moneda asociada a cada usuario.

```
SELECT
    u.user_id,
    u.nombre,
    u.saldo,
    m.currency_name,
    m.currency_symbol
FROM usuario AS u
INNER JOIN moneda AS m
    ON u.currency_id = m.currency_id;
```

```
mysql> SELECT u.user_id, u.nombre, u.saldo, m.currency_name, m.currency_symbol FROM usuario AS u INNER JOIN moneda AS m ON u.currency_id = m.currency_id;
+-----+-----+-----+-----+-----+
| user_id | nombre      | saldo   | currency_name | currency_symbol |
+-----+-----+-----+-----+-----+
| 1 | Angelica Peña | 850000.00 | Peso Chileno | $               |
| 2 | Juan Perez   | 1200.00  | Dolar Estadounidense | US$            |
| 3 | Camila Soto  | 950.00   | Euro          | EUR             |
+-----+-----+-----+-----+-----+
3 rows in set (0,11 sec)

mysql> SELECT u.nombre, m.currency_name FROM usuario AS u INNER JOIN moneda AS m ON u.currency_id = m.currency_id WHERE u.user_id = 1;
+-----+-----+
| nombre      | currency_name |
+-----+-----+
| Angelica Peña | Peso Chileno |
+-----+-----+
1 row in set (0,00 sec)

mysql>
```

Figura 5. INNER JOIN entre usuario y moneda. Fuente: elaboración propia.

5.5 Registro de transacciones

```
INSERT INTO transaccion
(sender_user_id, receiver_user_id, currency_id, importe)
VALUES
(1, 2, 1, 50000.00),
(2, 3, 2, 100.00),
(3, 1, 3, 50.00);

SELECT * FROM transaccion;
```

```
mysql> INSERT INTO transaccion (sender_user_id, receiver_user_id, currency_id, importe) VALUES (1, 2, 1, 50000.00), (2, 3, 2, 100.00), (3, 1, 3, 50.00);
Query OK, 3 rows affected (0,19 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM transaccion;
+-----+-----+-----+-----+-----+-----+
| transaction_id | sender_user_id | receiver_user_id | currency_id | importe | transaction_date |
+-----+-----+-----+-----+-----+-----+
| 1 | 1 | 2 | 1 | 50000.00 | 2026-09-06 19:59:01 |
| 2 | 2 | 3 | 2 | 100.00 | 2026-09-06 19:59:01 |
| 3 | 3 | 1 | 3 | 50.00 | 2026-09-06 19:59:01 |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0,00 sec)

mysql>
```

Figura 6. Transacciones registradas. Fuente: elaboración propia.

5.6 Consulta completa de transacciones

```
SELECT
    t.transaction_id,
    emisor.nombre AS emisor,
    receptor.nombre AS receptor,
    t.importe,
    m.currency_name AS moneda,
    m.currency_symbol AS simbolo,
    t.transaction_date
FROM transaccion AS t
INNER JOIN usuario AS emisor
    ON t.sender_user_id = emisor.user_id
INNER JOIN usuario AS receptor
    ON t.receiver_user_id = receptor.user_id
INNER JOIN moneda AS m
    ON t.currency_id = m.currency_id;
```

```
mysql> SELECT t.transaction_id, emisor.nombre AS emisor, receptor.nombre AS receptor, t.importe, m.currency_name AS moneda, m.currency_symbol AS simbolo, t.transaction_date FROM transaccion AS t INNER JOIN usuario AS emisor ON t.sender_user_id = emisor.user_id INNER JOIN usuario AS receptor ON t.receiver_user_id = receptor.user_id INNER JOIN moneda AS m ON t.currency_id = m.currency_id;
```

transaction_id	emisor	receptor	importe	moneda	simbolo	transaction_date
1	Angelica Peña	Juan Perez	50000.00	Peso Chileno	\$	2026-09-06 19:59:01
2	Juan Perez	Camila Soto	100.00	Dolar Estadounidense	US\$	2026-09-06 19:59:01
3	Camila Soto	Angelica Peña	50.00	Euro	EUR	2026-09-06 19:59:01

```
3 rows in set (0,18 sec)

mysql>
```

Figura 7. JOIN de usuario emisor, usuario receptor y moneda. Fuente: elaboración propia.

5.7 Transacciones de un usuario específico

```
WHERE t.sender_user_id = 1
OR t.receiver_user_id = 1;
```

La condición OR permite recuperar tanto los movimientos enviados como los recibidos por el usuario seleccionado.

```
mysql> SELECT t.transaction_id, emisor.nombre AS emisor, receptor.nombre AS receptor, t.importe, m.currency_name AS moneda, t.transaction_date FROM transaccion AS t INNER JOIN usuario AS emisor ON t.sender_user_id = emisor.user_id INNER JOIN usuario AS receptor ON t.receiver_user_id = receptor.user_id INNER JOIN moneda AS m ON t.currency_id = m.currency_id WHERE t.sender_user_id = 1 OR t.receiver_user_id = 1;
```

transaction_id	emisor	receptor	importe	moneda	transaction_date
1	Angelica Peña	Juan Perez	50000.00	Peso Chileno	2026-09-06 19:59:01
3	Camila Soto	Angelica Peña	50.00	Euro	2026-09-06 19:59:01

```
2 rows in set (0,08 sec)

mysql>
```

Figura 8. Transacciones en las que participa el usuario con user_id = 1. Fuente: elaboración propia.

5.8 Modificación de datos con UPDATE

```
UPDATE usuario
SET correo = 'angelica.pena@email.com'
WHERE user_id = 1;
```

```
mysql> SELECT user_id, nombre, correo FROM usuario WHERE user_id = 1;
```

user_id	nombre	correo
1	Angelica Peña	angelica@email.com

```
1 row in set (0,00 sec)

mysql> UPDATE usuario SET correo = 'angelica.pena@email.com' WHERE user_id = 1;
Query OK, 1 row affected (0,29 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> SELECT user_id, nombre, correo FROM usuario WHERE user_id = 1;
```

user_id	nombre	correo
1	Angelica Peña	angelica.pena@email.com

```
1 row in set (0,01 sec)
```

Figura 9. Actualización del correo de un usuario específico. Fuente: elaboración propia.

5.9 Eliminación de una transacción con DELETE

```
DELETE FROM transaccion
WHERE transaction_id = 2;
```

```
mysql> SELECT * FROM transaccion WHERE transaction_id = 2;
+-----+-----+-----+-----+-----+-----+
| transaction_id | sender_user_id | receiver_user_id | currency_id | importe | transaction_date |
+-----+-----+-----+-----+-----+-----+
| 2 | 2 | 3 | 2 | 100.00 | 2026-09-06 19:59:01 |
+-----+-----+-----+-----+-----+
1 row in set (0,01 sec)

mysql> DELETE FROM transaccion WHERE transaction_id = 2;
Query OK, 1 row affected (0,15 sec)

mysql> SELECT * FROM transaccion;
+-----+-----+-----+-----+-----+-----+
| transaction_id | sender_user_id | receiver_user_id | currency_id | importe | transaction_date |
+-----+-----+-----+-----+-----+-----+
| 1 | 1 | 2 | 1 | 50000.00 | 2026-09-06 19:59:01 |
| 3 | 3 | 1 | 3 | 50.00 | 2026-09-06 19:59:01 |
+-----+-----+-----+-----+-----+
2 rows in set (0,01 sec)
```

Figura 10. Eliminación de una fila de la tabla transaccion. Fuente: elaboración propia.

6. Transaccionalidad e integridad

6.1 Transferencia confirmada con COMMIT

```
START TRANSACTION;

UPDATE usuario
SET saldo = saldo - 20000.00
WHERE user_id = 1;

UPDATE usuario
SET saldo = saldo + 20000.00
WHERE user_id = 4;

INSERT INTO transaccion
(sender_user_id, receiver_user_id, currency_id, importe)
VALUES (1, 4, 1, 20000.00);

COMMIT;
```

La transferencia descuenta el monto al emisor, lo suma al receptor y registra el movimiento. COMMIT confirma los cambios como una sola operación lógica.

6.2 Error de integridad referencial y ROLLBACK

```
START TRANSACTION;

UPDATE usuario
SET saldo = saldo - 100000.00
WHERE user_id = 1;

INSERT INTO transaccion
(sender_user_id, receiver_user_id, currency_id, importe)
VALUES (1, 99, 1, 10000.00);

ROLLBACK;
```

La inserción falla porque receiver_user_id = 99 no existe en usuario. La clave foránea bloquea la operación y ROLLBACK restaura el saldo original. Esta prueba evidencia integridad referencial y atomicidad.

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0,00 sec)

mysql> UPDATE usuario SET saldo = saldo - 100000.00 WHERE user_id = 1;
Query OK, 1 row affected (0,02 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> SELECT user_id, nombre, saldo FROM usuario WHERE user_id = 1;
+-----+-----+-----+
| user_id | nombre      | saldo      |
+-----+-----+-----+
|      1 | Angelica Peña | 730000.00 |
+-----+-----+-----+
1 row in set (0,00 sec)

mysql> INSERT INTO transaccion (sender_user_id, receiver_user_id, currency_id, importe) VALUES (1, 99, 1, 10000.00);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails ('AlkeWallet`.`transaccion`, CONSTRAINT `transaccion_ibfk_2' FOREIGN KEY (`receiver_user_id`) REFERENCES `usuario` (`user_id`))
mysql> ROLLBACK;
Query OK, 0 rows affected (0,07 sec)

mysql> SELECT user_id, nombre, saldo FROM usuario WHERE user_id = 1;
+-----+-----+-----+
| user_id | nombre      | saldo      |
+-----+-----+-----+
|      1 | Angelica Peña | 830000.00 |
+-----+-----+-----+
1 row in set (0,00 sec)
```

Figura 11. Error de clave foránea y recuperación del saldo mediante ROLLBACK. Fuente: elaboración propia.

7. Consultas avanzadas y optimización

7.1 COUNT, SUM y GROUP BY

```
SELECT COUNT(*) AS total_transacciones
FROM transaccion;

SELECT SUM(importe) AS importe_total
FROM transaccion;

SELECT
    m.currency_name AS moneda,
    COUNT(t.transaction_id) AS cantidad_transacciones,
    SUM(t.importe) AS total_movido
FROM transaccion AS t
INNER JOIN moneda AS m
    ON t.currency_id = m.currency_id
GROUP BY m.currency_id, m.currency_name;
```

La suma global de importes se utilizó solo como demostración técnica de SUM, ya que el conjunto contiene monedas diferentes. El GROUP BY por moneda permite obtener resultados financieramente más interpretables.

```
mysql> SELECT COUNT(*) AS total_transacciones FROM transaccion;
+-----+
| total_transacciones |
+-----+
| 3 |
+-----+
1 row in set (0,09 sec)

mysql> SELECT SUM(importe) AS importe_total FROM transaccion;
+-----+
| importe_total |
+-----+
| 70050.00 |
+-----+
1 row in set (0,04 sec)

mysql> SELECT m.currency_name AS moneda, COUNT(t.transaction_id) AS cantidad_transacciones, SUM(t.importe) AS total_movido FROM
M transaccion AS t INNER JOIN moneda AS m ON t.currency_id = m.currency_id GROUP BY m.currency_id, m.currency_name;
+-----+-----+-----+
| moneda | cantidad_transacciones | total_movido |
+-----+-----+-----+
| Peso Chileno | 2 | 70000.00 |
| Euro | 1 | 50.00 |
+-----+-----+-----+
2 rows in set (0,06 sec)
```

Figura 12. Funciones de agregación COUNT, SUM y GROUP BY. Fuente: elaboración propia.

7.2 Subconsulta: total de transacciones por usuario

```
SELECT
    u.user_id,
    u.nombre,
    (
        SELECT COUNT(*)
        FROM transaccion AS t
        WHERE t.sender_user_id = u.user_id
            OR t.receiver_user_id = u.user_id
    ) AS total_transacciones
FROM usuario AS u;
```

```
mysql> SELECT u.user_id, u.nombre, ( SELECT COUNT(*) FROM transaccion AS t WHERE t.sender_user_id = u.user_id OR t.receiver_user_id = u.user_id ) AS total_transacciones FROM usuario AS u;
+-----+-----+-----+
| user_id | nombre      | total_transacciones |
+-----+-----+-----+
| 1 | Angelica Peña | 3 |
| 2 | Juan Perez   | 1 |
| 3 | Camila Soto  | 1 |
| 4 | Pedro Gonzales | 1 |
+-----+-----+-----+
4 rows in set (0,07 sec)
```

Figura 13. Subconsulta para contar transacciones por usuario. Fuente: elaboración propia.

7.3 Índices compuestos

```
CREATE INDEX idx_transaccion_emisor_fecha
ON transaccion (sender_user_id, transaction_date);

CREATE INDEX idx_transaccion_receptor_fecha
ON transaccion (receiver_user_id, transaction_date);
```

Los índices compuestos apoyan búsquedas por usuario y fecha en una tabla de transacciones que, en un sistema real, puede crecer considerablemente.

```
Query OK, 0 rows affected (2,16 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> SHOW INDEX FROM transaccion;
+-----+-----+-----+-----+-----+-----+-----+-----+
| Table | Non_unique | Key_name | Seq_in_index | Column_name | Collation | Cardinality | Sub_ |
| part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+-----+-----+-----+-----+-----+-----+-----+-----+
| transaccion | 0 | PRIMARY | 1 | transaction_id | A | 3 | |
| NULL | NULL | | BTREE | | | YES | NULL |
| transaccion | 1 | currency_id | 1 | currency_id | A | 2 |
| NULL | NULL | | BTREE | | | YES | NULL |
| transaccion | 1 | idx_transaccion_emisor_fecha | 1 | sender_user_id | A | 2 |
| NULL | NULL | | BTREE | | | YES | NULL |
| transaccion | 1 | idx_transaccion_emisor_fecha | 2 | transaction_date | A | 3 |
| NULL | NULL | | BTREE | | | YES | NULL |
| transaccion | 1 | idx_transaccion_receptor_fecha | 1 | receiver_user_id | A | 3 |
| NULL | NULL | | BTREE | | | YES | NULL |
| transaccion | 1 | idx_transaccion_receptor_fecha | 2 | transaction_date | A | 3 |
| NULL | NULL | | BTREE | | | YES | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+
6 rows in set (1,16 sec)
```

Figura 14. Verificación de índices de la tabla transaccion. Fuente: elaboración propia.

7.4 ALTER TABLE: fecha de creación

```
ALTER TABLE usuario
ADD COLUMN fecha_creacion DATETIME
NOT NULL DEFAULT CURRENT_TIMESTAMP;
```

```
mysql> DESCRIBE usuario;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| user_id | int | NO | PRI | NULL | auto_increment |
| nombre | varchar(100) | NO | | NULL | |
| correo | varchar(150) | NO | UNI | NULL | |
| contrasena | varchar(255) | NO | | NULL | |
| saldo | decimal(12,2) | NO | | 0.00 | |
| currency_id | int | NO | MUL | NULL | |
| fecha_creacion | datetime | NO | | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+-----+-----+-----+-----+-----+-----+
7 rows in set (0,04 sec)

mysql> SELECT user_id, nombre, fecha_creacion FROM usuario;
+-----+-----+-----+
| user_id | nombre | fecha_creacion |
+-----+-----+-----+
| 1 | Angelica Peña | 2026-09-06 22:28:42 |
| 2 | Juan Perez | 2026-09-06 22:28:42 |
| 3 | Camila Soto | 2026-09-06 22:28:42 |
| 4 | Pedro Gonzales | 2026-09-06 22:28:42 |
+-----+-----+-----+
4 rows in set (0,00 sec)
```

Figura 15. Columna fecha_creacion agregada a usuario. Fuente: elaboración propia.

7.5 Vista Top 5 de usuarios por saldo

```
CREATE VIEW top_5_usuarios_saldo AS
SELECT
    user_id,
    nombre,
    saldo
FROM usuario
ORDER BY saldo DESC
LIMIT 5;
```

La vista almacena la consulta lógica y refleja los datos actuales de la tabla usuario sin duplicarlos físicamente.

```
mysql> SHOW TABLES;
+-----+-----+
| Tables_in_AlkeWallet | Table_type |
+-----+-----+
| moneda | BASE TABLE |
| top_5_usuarios_saldo | VIEW |
| transaccion | BASE TABLE |
| usuario | BASE TABLE |
+-----+-----+
4 rows in set (0,02 sec)

mysql> SHOW CREATE TABLE top_5_usuarios_saldo;
+-----+-----+-----+-----+-----+-----+
| View | Create View |
+-----+-----+-----+-----+-----+-----+
| top_5_usuarios_saldo | CREATE ALGORITHM=UNDEFINED DEFINER='root'@'localhost' SQL SECURITY DEFINER VIEW 'top_5_usuarios_saldo' AS select 'usuario`.`user_id' AS 'user_id','usuario`.`nombre' AS 'nombre','usuario`.`saldo' AS 'saldo' from 'usuario' order by 'usuario`.`saldo' desc limit 5 |
+-----+-----+-----+-----+-----+-----+
```

Figura 16. Verificación de top_5_usuarios_saldo como VIEW. Fuente: elaboración propia.

8. Prueba final del proyecto

El script final fue preparado para reconstruir el proyecto desde cero. Antes de la prueba se generó un respaldo de la base; posteriormente se ejecutó `AlkeWallet_Final.sql` desde la terminal y se guardó el registro de salida en `prueba_final.txt`.

```
sudo mysqldump AlkeWallet > AlkeWallet_backup.sql
sudo mysql < AlkeWallet_Final.sql 2>&1 | tee prueba_final.txt
grep -i "error" prueba_final.txt
```

La ejecución final completó la creación de tablas, datos, índices y vista. La revisión del registro no mostró errores no controlados.

```
ange@ange-Lenovo-V110-14IAP:~/Escritorio/Alke-Wallet v2/AlkeWallet-BaseDatos$ sudo mysql < AlkeWallet_Final.sql 2>&1 |
tee prueba_final.txt
70050.00
moneda cantidad_transacciones total_movido
Peso Chileno 2 70000.00
Euro 1 50.00
user_id nombre total_transacciones
1 Angelica Peña 3
2 Juan Perez 1
3 Camila Soto 1
4 Pedro Gonzales 1
Table Non unique Key_name Seq_in_index Column name Collation Cardinality Sub_part P
acked Null Index_type Index Comment Index comment Visible Expression
transaccion 0 PRIMARY 1 transaction_id A 1 NULL NULL BTREE Y
ES NULL
transaccion 1 fk_transaccion_moneda 1 currency_id A 1 NULL NULL BTREEY
ES NULL
transaccion 1 idx_transaccion_emisor_fecha 1 sender_user_id A 1 NULL NULL B
TREE YES NULL
transaccion 1 idx_transaccion_emisor_fecha 2 transaction_date A 1 NULL NULL B
TREE YES NULL
transaccion 1 idx_transaccion_receptor_fecha 1 receiver_user_id A 1 NULL NULL B
TREE YES NULL
transaccion 1 idx_transaccion_receptor_fecha 2 transaction_date A 1 NULL NULL B
TREE YES NULL
user_id nombre saldo
1 Angelica Peña 830000.00
4 Pedro Gonzales 120000.00
2 Juan Perez 1200.00
3 Camila Soto 950.00
Tables in AlkeWallet Table_type
moneda BASE TABLE
top_5_usuarios_saldo VIEW
transaccion BASE TABLE
usuario BASE TABLE
ange@ange-Lenovo-V110-14IAP:~/Escritorio/Alke-Wallet v2/AlkeWallet-BaseDatos$
```

Figura 17. Prueba final de ejecución del script `AlkeWallet_Final.sql`. Fuente: elaboración propia.

9. Verificación de requisitos

Requisito	Estado	Evidencia
Crear base AlkeWallet	Cumplido	CREATE DATABASE / Figura 1
Tablas usuario, moneda y transaccion	Cumplido	DDL / Figura 2
Claves primarias y foráneas	Cumplido	DESCRIBE y modelo ER
Insertar datos de prueba	Cumplido	Figuras 3 y 5
Consulta moneda de un usuario	Cumplido	INNER JOIN / Figura 4
Todas las transacciones	Cumplido	Figura 5
Transacciones de usuario específico	Cumplido	Figura 7
Modificar correo	Cumplido	UPDATE / Figura 8
Eliminar transacción	Cumplido	DELETE / Figura 9
COMMIT y ROLLBACK	Cumplido	Sección 6 / Figura 10
COUNT, SUM y GROUP BY	Cumplido	Figura 11
Subconsulta	Cumplido	Figura 12
Índices compuestos	Cumplido	Figura 13
ALTER TABLE	Cumplido	Figura 14
Diagrama ER y 3FN	Cumplido	Figura 15 y sección 4.5
Vista Top 5	Cumplido (Plus)	Figura 16
Script final probado	Cumplido	Figura 17

10. Conclusiones

El proyecto permitió construir una base de datos relacional funcional para Alke Wallet y aplicar de manera integrada los contenidos del módulo. La estructura separa correctamente usuarios, monedas y transacciones, evitando redundancia y utilizando claves foráneas para mantener la integridad.

Las operaciones DDL y DML se complementaron con consultas JOIN, agregaciones, subconsultas e índices. La prueba con COMMIT y ROLLBACK permitió comprobar cómo una base de datos puede mantener la coherencia cuando una operación compuesta falla.

Finalmente, el script completo fue ejecutado desde cero en MySQL 8, verificando que la base, sus tablas, índices y vista pueden reconstruirse de forma reproducible.

Fuente de requerimientos

Consigna del proyecto “Alke Wallet” – Evaluación del módulo: Fundamentos de Bases de Datos Relacionales (Alkemy). Las capturas, el script SQL y los resultados presentados corresponden a la implementación realizada durante el desarrollo del proyecto.

Anexo A. Script SQL completo

El siguiente script corresponde a la versión final utilizada para reconstruir Alke Wallet. La sentencia que provoca el error de integridad referencial se mantiene comentada para que la demostración de ROLLBACK pueda ejecutarse manualmente.

```
-- =====
-- PROYECTO: ALKE WALLET V2
-- MÓDULO: FUNDAMENTOS DE BASES DE DATOS RELACIONALES
-- Motor utilizado: MySQL 8
-- =====

-- 1. CREAR BASE DE DATOS
DROP DATABASE IF EXISTS AlkeWallet;
CREATE DATABASE AlkeWallet;
USE AlkeWallet;

-- 2. VERIFICAR CREACIÓN
SHOW DATABASES;

-- 3. CREAR TABLA MONEDA
CREATE TABLE moneda (
    currency_id INT AUTO_INCREMENT PRIMARY KEY,
    currency_name VARCHAR(50) NOT NULL,
    currency_symbol VARCHAR(10) NOT NULL
);

-- 4. CREAR TABLA USUARIO
CREATE TABLE usuario (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    correo VARCHAR(150) NOT NULL UNIQUE,
    contrasena VARCHAR(255) NOT NULL,
    saldo DECIMAL(12,2) NOT NULL DEFAULT 0.00,
    currency_id INT NOT NULL,

    CONSTRAINT fk_usuario_moneda
        FOREIGN KEY (currency_id)
        REFERENCES moneda(currency_id)
);

-- 5. CREAR TABLA TRANSACCION
CREATE TABLE transaccion (
    transaction_id INT AUTO_INCREMENT PRIMARY KEY,
    sender_user_id INT NOT NULL,
    receiver_user_id INT NOT NULL,
    currency_id INT NOT NULL,
    importe DECIMAL(12,2) NOT NULL,
    transaction_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT fk_transaccion_emisor
        FOREIGN KEY (sender_user_id)
        REFERENCES usuario(user_id),

    CONSTRAINT fk_transaccion_receptor
        FOREIGN KEY (receiver_user_id)
        REFERENCES usuario(user_id),

    CONSTRAINT fk_transaccion_moneda
```

```

        FOREIGN KEY (currency_id)
        REFERENCES moneda(currency_id)
    );

-- 6. VERIFICAR TABLAS
SHOW TABLES;
DESCRIBE moneda;
DESCRIBE usuario;
DESCRIBE transaccion;

-- 7. INSERTAR MONEDAS
INSERT INTO moneda
(currency_name, currency_symbol)
VALUES
('Peso Chileno', '$'),
('Dolar Estadounidense', 'US$'),
('Euro', 'EUR');

-- 8. CONSULTAR MONEDAS
SELECT * FROM moneda;

-- 9. INSERTAR USUARIOS
INSERT INTO usuario
(nombre, correo, contrasena, saldo, currency_id)
VALUES
('Angelica Peña', 'angelica@email.com', 'clave123', 850000.00, 1),
('Juan Perez', 'juan@email.com', 'clave456', 1200.00, 2),
('Camila Soto', 'camila@email.com', 'clave789', 950.00, 3);

-- 10. CONSULTAR USUARIOS
SELECT * FROM usuario;

-- 11. INNER JOIN USUARIO - MONEDA
SELECT
    u.user_id,
    u.nombre,
    u.saldo,
    m.currency_name,
    m.currency_symbol
FROM usuario AS u
INNER JOIN moneda AS m
    ON u.currency_id = m.currency_id;

-- 12. MONEDA DE UN USUARIO ESPECÍFICO
SELECT
    u.nombre,
    m.currency_name
FROM usuario AS u
INNER JOIN moneda AS m
    ON u.currency_id = m.currency_id
WHERE u.user_id = 1;

-- 13. INSERTAR TRANSACCIONES DE PRUEBA
INSERT INTO transaccion
(sender_user_id, receiver_user_id, currency_id, importe)
VALUES
(1, 2, 1, 50000.00),
(2, 3, 2, 100.00),
(3, 1, 3, 50.00);

-- 14. CONSULTAR TODAS LAS TRANSACCIONES
SELECT * FROM transaccion;

```

```

-- 15. TRANSACCIONES CON INFORMACIÓN COMPLETA
SELECT
    t.transaction_id,
    emisor.nombre AS emisor,
    receptor.nombre AS receptor,
    t.importe,
    m.currency_name AS moneda,
    m.currency_symbol AS simbolo,
    t.transaction_date
FROM transaccion AS t
INNER JOIN usuario AS emisor
    ON t.sender_user_id = emisor.user_id
INNER JOIN usuario AS receptor
    ON t.receiver_user_id = receptor.user_id
INNER JOIN moneda AS m
    ON t.currency_id = m.currency_id;

-- 16. TRANSACCIONES DE UN USUARIO ESPECÍFICO
SELECT
    t.transaction_id,
    emisor.nombre AS emisor,
    receptor.nombre AS receptor,
    t.importe,
    m.currency_name AS moneda,
    t.transaction_date
FROM transaccion AS t
INNER JOIN usuario AS emisor
    ON t.sender_user_id = emisor.user_id
INNER JOIN usuario AS receptor
    ON t.receiver_user_id = receptor.user_id
INNER JOIN moneda AS m
    ON t.currency_id = m.currency_id
WHERE t.sender_user_id = 1
    OR t.receiver_user_id = 1;

-- 17. MODIFICAR CORREO
UPDATE usuario
SET correo = 'angelica.pena@email.com'
WHERE user_id = 1;

SELECT user_id, nombre, correo
FROM usuario
WHERE user_id = 1;

-- 18. ELIMINAR UNA TRANSACCIÓN
SELECT *
FROM transaccion
WHERE transaction_id = 2;

DELETE FROM transaccion
WHERE transaction_id = 2;

SELECT * FROM transaccion;

-- 19. AGREGAR FECHA DE CREACIÓN
ALTER TABLE usuario
ADD COLUMN fecha_creacion DATETIME
NOT NULL DEFAULT CURRENT_TIMESTAMP;

DESCRIBE usuario;

```

```

-- 20. INSERTAR SEGUNDO USUARIO EN CLP
INSERT INTO usuario
(nombre, correo, contrasena, saldo, currency_id)
VALUES
('Pedro Gonzales', 'pedro@email.com', 'clave101', 100000.00, 1);

-- 21. TRANSACCIÓN CONTROLADA CON COMMIT
START TRANSACTION;

UPDATE usuario
SET saldo = saldo - 20000.00
WHERE user_id = 1;

UPDATE usuario
SET saldo = saldo + 20000.00
WHERE user_id = 4;

INSERT INTO transaccion
(sender_user_id, receiver_user_id, currency_id, importe)
VALUES
(1, 4, 1, 20000.00);

SELECT user_id, nombre, saldo
FROM usuario
WHERE user_id IN (1, 4);

COMMIT;

-- 22. SIMULACIÓN DE ERROR Y ROLLBACK
START TRANSACTION;

UPDATE usuario
SET saldo = saldo - 100000.00
WHERE user_id = 1;

SELECT user_id, nombre, saldo
FROM usuario
WHERE user_id = 1;

-- Error intencional para demostración manual:
-- INSERT INTO transaccion
-- (sender_user_id, receiver_user_id, currency_id, importe)
-- VALUES
-- (1, 99, 1, 10000.00);

ROLLBACK;

SELECT user_id, nombre, saldo
FROM usuario
WHERE user_id = 1;

-- 23. FUNCIONES DE AGREGACIÓN
SELECT COUNT(*) AS total_transacciones
FROM transaccion;

SELECT SUM(importe) AS importe_total
FROM transaccion;

-- 24. AGRUPAR TRANSACCIONES POR MONEDA
SELECT
    m.currency_name AS moneda,

```

```

        COUNT(t.transaction_id) AS cantidad_transacciones,
        SUM(t.importe) AS total_movido
FROM transaccion AS t
INNER JOIN moneda AS m
    ON t.currency_id = m.currency_id
GROUP BY
    m.currency_id,
    m.currency_name;

```

-- 25. SUBCONSULTA: TOTAL DE TRANSACCIONES POR USUARIO

```

SELECT
    u.user_id,
    u.nombre,
    (
        SELECT COUNT(*)
        FROM transaccion AS t
        WHERE t.sender_user_id = u.user_id
            OR t.receiver_user_id = u.user_id
    ) AS total_transacciones
FROM usuario AS u;

```

-- 26. CREAR ÍNDICES COMPUESTOS

```

CREATE INDEX idx_transaccion_emisor_fecha
ON transaccion
(sender_user_id, transaction_date);

```

```

CREATE INDEX idx_transaccion_receptor_fecha
ON transaccion
(receiver_user_id, transaction_date);

```

```

SHOW INDEX FROM transaccion;

```

-- 27. CREAR VISTA TOP 5

```

CREATE VIEW top_5_usuarios_saldo AS
SELECT
    user_id,
    nombre,
    saldo
FROM usuario
ORDER BY saldo DESC
LIMIT 5;

```

-- 28. CONSULTAR VISTA

```

SELECT * FROM top_5_usuarios_saldo;

```

-- 29. VERIFICAR TABLAS Y VISTAS

```

SHOW FULL TABLES;

```